

Workflow de Modelagem de Software

Do requisito ao schema SQL — CRUD de Produtos

Documento Didático

Cinco etapas encadeadas: Caso de Uso · Robustez ·
Classes Conceitual · Modelo Físico · Schema SQL.

Sumário

Introdução · O exemplo único

Capítulo 1 · Diagrama de Casos de Uso

Capítulo 2 · Diagrama de Robustez

Capítulo 3 · Diagrama de Classes Conceitual

Capítulo 4 · Modelo Físico de Dados (Crow's Foot)

Capítulo 5 · Schema SQL

Rastreabilidade · Do requisito à linha de SQL

Apêndice · Quadro comparativo das cinco etapas

Introdução · O exemplo único

Por que um workflow?

Modelagem de software raramente é um único diagrama: é uma **cadeia** em que cada modelo responde a perguntas que o anterior lançou, e deixa ganchos explícitos para o próximo. Este documento apresenta cinco modelos encadeados — caso de uso, robustez, classes conceitual, modelo físico e schema SQL — usando **um único exemplo consistente**: o CRUD de Produtos, com atributos **id**, **nome**, **estoque** e **preço**, e métodos **criarProduto**, **listarProdutos**, **buscarProdutoPorId**, **atualizarProduto** e **removerProduto**.

Atributos de Produto

- **id** — numérico, auto-incrementável, único.
- **nome** — texto curto obrigatório (até 120 caracteres).
- **estoque** — inteiro não-negativo.
- **preço** — decimal (12,2) não-negativo.

Operações de CRUD

- `criarProduto()` · `listarProdutos()` · `buscarProdutoPorId(id)`
- `atualizarProduto(id, dados)` · `removerProduto(id)`

Em cada capítulo a seguir você verá o mesmo conceito ganhar uma nova camada de detalhe — sempre no estilo da referência visual indicada.

Diagrama de Casos de Uso

A voz do usuário: quem faz o quê com o sistema.

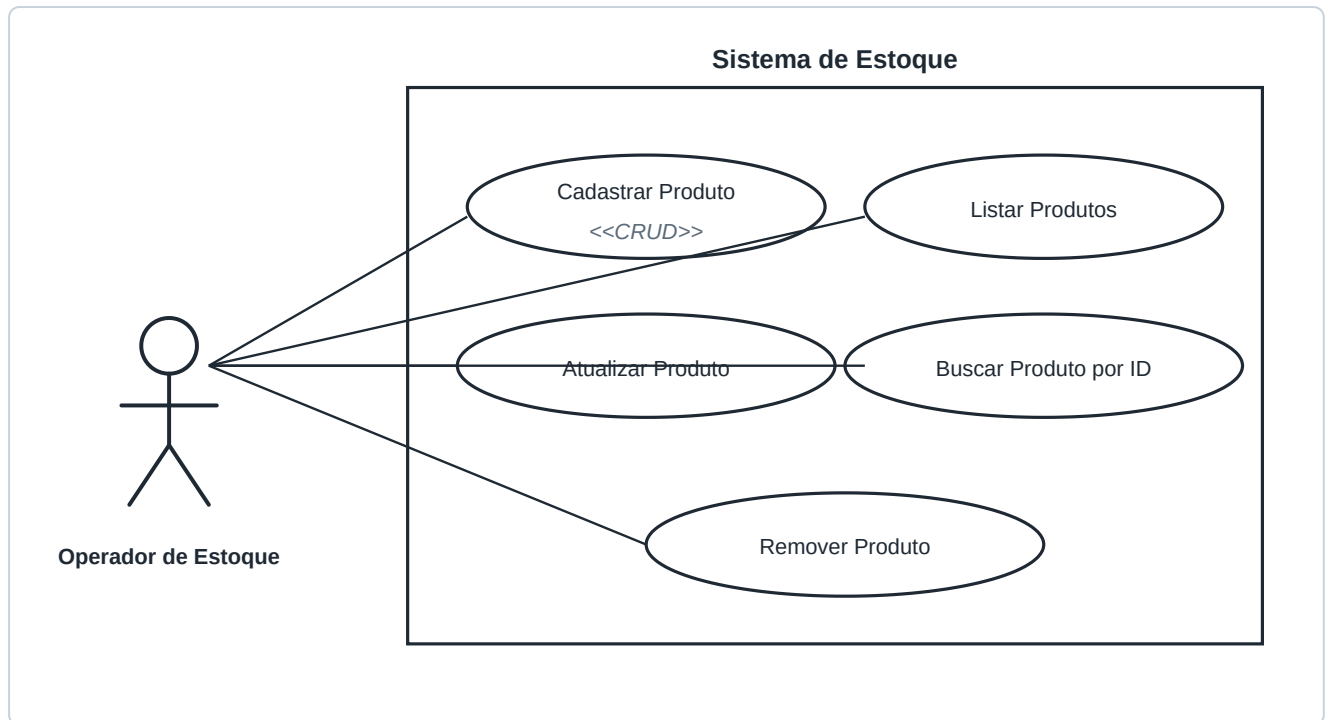


Figura 1 · Estilo referência anexo 1 — traço preto definido, fundo branco puro.

Decisões de modelagem

- **Um único ator** (Operador de Estoque) — todo o CRUD fica sob sua responsabilidade.
- **Cinco casos de uso**, cada um espelha um método do CRUD; nenhum depende via <<include>>, mas Atualizar e Remover *estendem* "Buscar por ID".
- **Nome dos casos iniciado por verbo** no infinitivo, conforme convenção UML.

BOAS PRÁTICAS

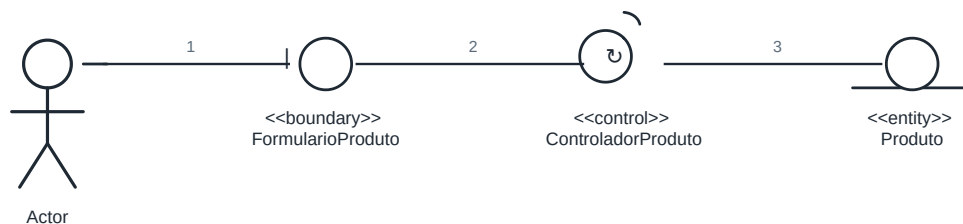
Nomeie casos de uso pelo **valor para o ator**. "Cadastrar Produto" entrega resultado observável (produto novo existe) — diferente de "preencher formulário".

Saída para a próxima etapa → Cada caso de uso gera um cenário; cada cenário vira sequência Boundary · Control · Entity no Capítulo 2 (Diagrama de Robustez).

Diagrama de Robustez

Dos casos de uso aos objetos por cenário — *Boundary · Control · Entity*.

Cenário 1 · Cadastrar Produto



Cenário 2 · Atualizar Produto

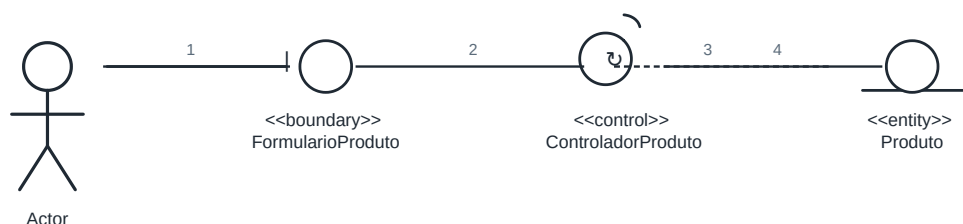


Figura 2 · Robustez com estereótipos *Boundary · Control · Entity* (referência anexo 2).

Decisões de modelagem

- **Um boundary polimórfico** (FormularioProduto) cobre cadastro e edição; evita proliferar telas.
- **Um control** (ControladorProduto) orquestra todos os cenários do agregado.
- **Entity carrega invariantes** (nome obrigatório, estoque ≥ 0 , preço ≥ 0) — preparo da classe do Cap. 3.

ERROS COMUNS / BOAS PRÁTICAS

Não atribua regras de negócio ao boundary (UI não deve validar preço ≥ 0) e não confunda entity com tabela — entity ainda é conceito, sem colunas.

Saída para a próxima etapa → **As entity do diagrama viram classes UML; os métodos identificados nos controls viram operações no Capítulo 3 (Classes Conceituais).**

Diagrama de Classes Conceitual

A classe *Produto* como vocabulário canônico do domínio.

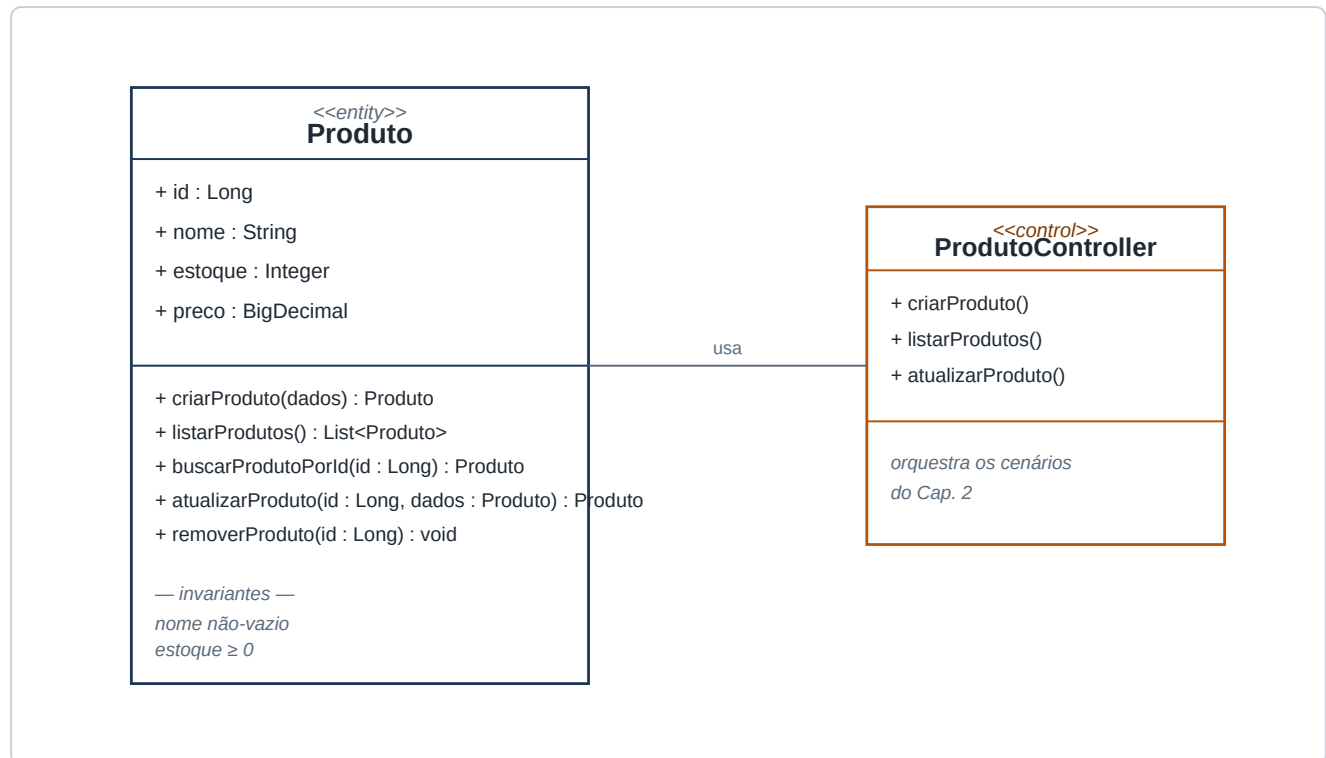


Figura 3 · UML clássico — classe `<<entity>>` *Produto* + `<<control>>` *ProdutoController*.

Decisões de modelagem

- **Tipos conceituais** (Long, String, Integer, BigDecimal) — o mapeamento para SQL fica no Cap. 4.
- **Operações do CRUD na classe** para fins didáticos — em projetos reais migrariam para um Service.
- **Invariantes de domínio** ficam anotadas; migram depois para CHECK no Cap. 5.

ERROS COMUNS / BOAS PRÁTICAS

Não usar tipos SQL na classe conceitual (contamina o modelo) e não omitir invariantes — sem elas, o banco aceita valores inválidos silenciosamente.

Saída para a próxima etapa → [Atributos viram colunas, tipos viram tipos físicos e invariantes viram restrições CHECK/NOT NULL no Capítulo 4 \(Modelo Físico\).](#)

Modelo Físico de Dados (Crow's Foot)

Da classe conceitual à tabela relacional executável.

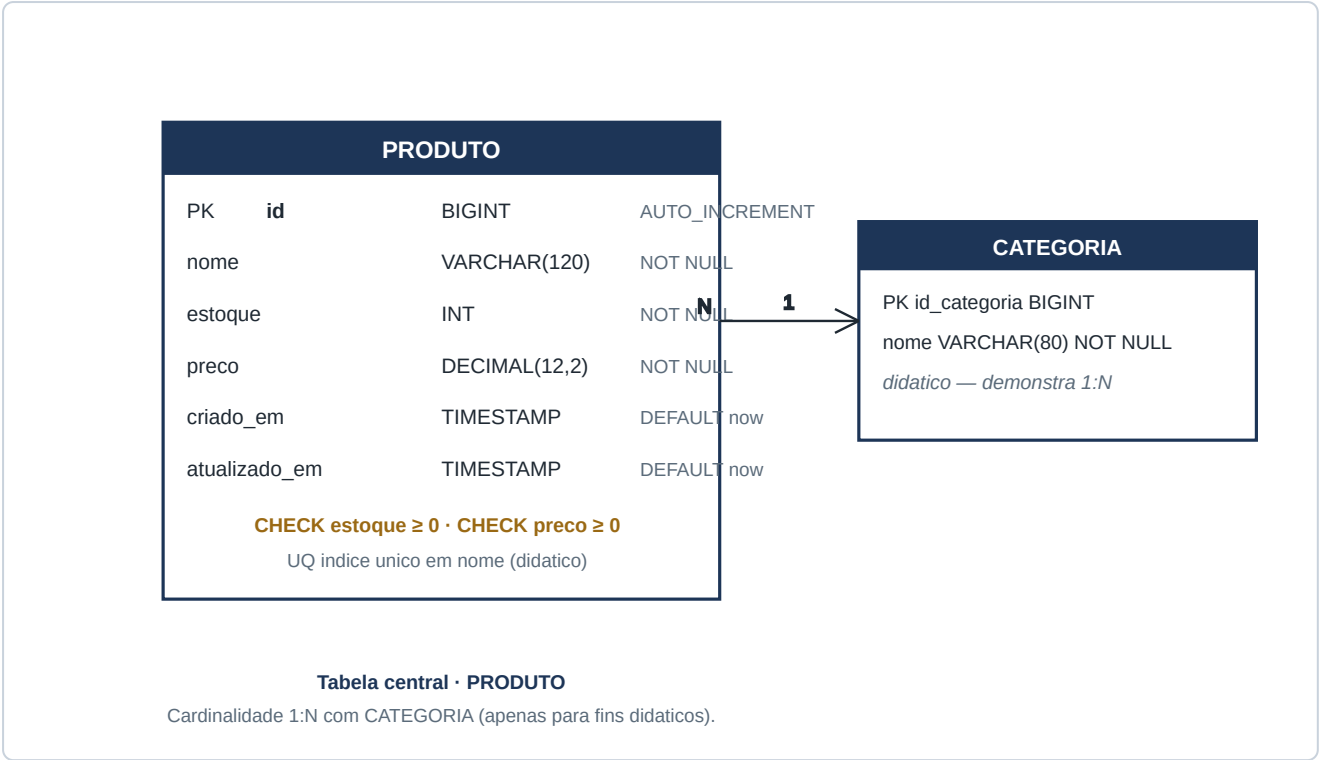


Figura 4 · Modelo Físico — notação Crow's Foot com tabela PRODUTO no centro.

Decisões de modelagem

- **DECIMAL(12,2) para preço** — não usar FLOAT/DOUBLE em valores monetários.
- **Restrições CHECK explícitas** para invariantes (estoque ≥ 0, preço ≥ 0).
- **Colunas de auditoria** (criado_em, atualizado_em) para rastreabilidade temporal.

ERROS COMUNS / BOAS PRÁTICAS

Use **DECIMAL** para moeda, nunca float. Leve invariantes para o banco — funciona como defesa em profundidade mesmo se a aplicação falhar.

Saída para a próxima etapa → A tabela produto vira um **CREATE TABLE** executável no Capítulo 5, acompanhado de **INSERT/SELECT/UPDATE/DELETE**.

Schema SQL

Materialização executável: DDL + operações do CRUD.

```
-- DDL: tabela produto + restrições + índice único (opcional)
CREATE TABLE produto (
  id          BIGINT          NOT NULL AUTO_INCREMENT,
  nome        VARCHAR(120)    NOT NULL,
  estoque     INT             NOT NULL,
  preco       DECIMAL(12,2)   NOT NULL,
  criado_em   TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT pk_produto
    PRIMARY KEY (id),
  CONSTRAINT ck_produto_estoque
    CHECK (estoque >= 0),
  CONSTRAINT ck_produto_preco
    CHECK (preco >= 0),
  CONSTRAINT uq_produto_nome
    UNIQUE (nome)
);

-- 1) criarProduto
INSERT INTO produto (nome, estoque, preco)
VALUES ('Caneta esferografica azul', 100, 2.50);

-- 2) listarProdutos / 3) buscarProdutoPorId
SELECT id, nome, estoque, preco FROM produto ORDER BY nome ASC;
SELECT id, nome, estoque, preco FROM produto WHERE id = 1;

-- 4) atualizarProduto
UPDATE produto
  SET estoque = 150, preco = 2.75, atualizado_em = CURRENT_TIMESTAMP
  WHERE id = 1;

-- 5) removerProduto
DELETE FROM produto WHERE id = 1;
```

Figura 5 · CREATE TABLE + INSERT + SELECT + UPDATE + DELETE coerentes com o CRUD.

Decisões & boas práticas

- **CHECK + NOT NULL** são a materialização SQL das invariantes do Cap. 3 — defesa em profundidade.
- **DECIMAL(12,2)** para preço; nunca **FLOAT** em moeda. Versione o schema como migrations.

Fim do workflow → [Próximos passos: migrations \(Flyway/Liquibase\), testes das constraints e seed inicial.](#)

Rastreabilidade · Do requisito à linha de SQL

O requisito "**Cadastrar Produto**" atravessa todas as cinco etapas até virar SQL executável. Esta tabela mostra a correspondência direta entre cada modelo.

Etapa	Artefato de Produto	Detalhe
Caso de Uso	Cadastrar Produto	Caso de uso no system boundary; ator Operador de Estoque.
Robustez	Boundary FormularioProduto Control ControladorProduto Entity Produto	Cenário 1: Actor → Boundary (1) → Control (2) → Entity (3) → retorno de confirmação (4).
Classes Conceitual	Classe Produto · método criarProduto()	Atributos: id: Long, nome: String, estoque: Integer, preco: BigDecimal.
Modelo Físico	Tabela produto	BIGINT PK AUTO_INCREMENT, VARCHAR(120) NOT NULL, INT CHECK ≥0, DECIMAL(12,2) CHECK ≥0.
Schema SQL	INSERT INTO produto (...)	Materialização executável da operação criarProduto(). O banco rejeita valores inválidos graças aos CHECK.

PONTO-CHAVE

Rastreabilidade permite identificar **onde** uma decisão se perdeu quando há quebra entre os modelos (ex.: coluna **nome** virar opcional no banco enquanto o caso de uso exige obrigatório).

Apêndice · Quadro comparativo das cinco etapas

Visão consolidada para consulta rápida. Cada linha mostra uma dimensão comparada entre os cinco modelos do workflow.

Dimensão	Caso de Uso	Robustez	Classes Conceitual	Modelo Físico	Schema SQL
Objetivo	Capturar intenções do ator.	Mostrar como objetos entram no cenário.	Fixar classes do domínio.	Definir tabelas, tipos e restrições.	Emitir DDL + DML executáveis.
Artefato	Diagrama com elipses e ator.	Diagrama Boundary / Control / Entity.	Classes com atributos e operações.	Tabelas Crow's Foot com colunas.	Scripts SQL versionados.
Pergunta-chave	"Quem faz o quê?"	"Quais objetos atuam?"	"Que informação existe?"	"Como vira tabela?"	"Qual script executa o CRUD?"
Exemplo · Produtos	Cadastrar, Listar, Buscar por ID, Atualizar, Remover.	FormularioProduto, ControladorProduto, Produto.	Classe Produto + 5 métodos CRUD.	Tabela produto + CHECK e NOT NULL.	CREATE TABLE + INSERT/SELECT/UPDATE/DELETE.

Glossário

Boundary · objeto de interface com o ator (tela, formulário).

Control · orquestrador do fluxo de um caso de uso.

Entity · conceito do domínio, geralmente persistido.

Crow's Foot · notação relacional com cardinalidade 1, N, 0..N.

DDL / DML · Data Definition Language (CREATE/ALTER) e Data Manipulation Language (INSERT/SELECT/UPDATE/DELETE).